1)

- Тип client_id можно поменять на int (этот тип обеспечивает быструю индексации, легко настраивается для автоинкремента, также обеспечивает эффективное использование памяти по сравнению с number) . Если он является первичным ключом, то нужно убедиться, что он автоматически инкрементируется и является уникальным.
- client_name и client_surname: можно ограничить длину до более разумного значения, например varchar(30), это поможет избежать избыточности использования места
- login: всё в порядке
- city_id: нужно убедиться, что существует таблица городов, на которую ссылается этот идентификатор
- age: Так как возраст меняется каждый год, то лучше хранить дату рождения клиента (типа date), что поможет вычислять возраст клиента на текущий момент
- reg_date: нужно проверить, что поле автоматически заполняется при регистрации клиента

2)

- В таблице items item_cost нужно заменить на current_item_cost(типа decimal(10,2) для лучшей точности цены), так как цены время от времени меняются, а нам нужно знать текущую стоимость товара.
- Тип item_id лучше поменять на int для облегчения настройки автоинкремента.
- Нужно добавить таблицу, которая будет хранить историю цен, например, item_price_history. Она будет состоять из:
    - item_id(int) - идентификатор товара, на который ссылается запись
    - price(decimal(10,2)) - стоимость товара на определенную дату. Хранит историческую стоимость товара
    - day_date(date) - дата, с которой начинает действовать данная стоимость
- Логика работы:
    - При изменении стоимости товара в таблице items обновляется поле current_item_cost, в таблицу item_price_history добавляется новая запись с item_id, новой стоимостью (price) и датой изменения (day_date).
    - Для построения отчетов за определенный период используется таблица item_price_history. Можно определить, какая стоимость была актуальна в любой момент времени, используя поле day_date.

3)

```
SELECT D.DEPARTMENT_ID,
       D.DEPARTMENT_NAME,
       SUM(E.SALARY) AS TOTAL_SALARY
FROM EMPLOYEES E
JOIN DEPARTMENTS D ON E.DEPARTMENT_ID = D.DEPARTMENT_ID
JOIN LOCATIONS L ON D.LOCATION_ID = L.LOCATION_ID
WHERE E.MANAGER_ID IS NULL
  AND L.CITY = 'Seoul'
GROUP BY D.DEPARTMENT_ID, D.DEPARTMENT_NAME
HAVING SUM(E.SALARY) >= 100000;
```

- Для соединения таблиц используем JOIN, это делает запрос более читаемым, а также он необходим для соединения с таблицей LOCATIONS

- Агрегатная функция SUM не может использоваться в WHERE, поэтому используем её в HAVING, ведь она применяется к результатам группировки
- Используем IS NULL для проверки отсутствия менеджера
- Группировка данных должна быть по DEPARTMENT_ID, а не только по DEPARTMENT_NAME, иначе данные могут агрегироваться некорректно

4) Если карандаш положили вплотную к стене, то как бы человек не старался, перепрыгнуть у него не получится